

Addressing Combinatorial Optimization with Estimation of Distribution Algorithms Based on Diffusion Models

Abstract

Diffusion models have demonstrated remarkable success in modeling high-dimensional probability distributions within machine learning. Their potential for modeling search distributions in combinatorial optimization, however, remains largely unexplored. This paper bridges this gap by integrating diffusion models into Estimation of Distribution Algorithms (EDAs). We propose two novel EDAs: a diffusion-by-denoising EDA (Diff-EDA) and a diffusion-by-deblending EDA (DbD-EDA), both adapted for discrete optimization. Key adaptations include the use of Gumbel-Softmax for discrete variables, fitness-guided sampling, and tailored loss functions. Through extensive experiments on benchmark additive functions and combinatorial problem instances (SAT, Ising, UBQP), we validate the effectiveness of the proposed algorithms. Our results show that diffusion-based EDAs can outperform classical EDAs based on probabilistic graphical models and contemporary neural-network-based EDAs, particularly on problems with complex variable interactions. This work establishes a new direction for EDAs, demonstrating that diffusion models can provide a powerful and flexible framework for learning and sampling from search distributions in evolutionary optimization.

Keywords

estimation of distribution algorithms, diffusion models, evolutionary algorithms, neural networks, search distributions

ACM Reference Format:

. 2026. Addressing Combinatorial Optimization with Estimation of Distribution Algorithms Based on Diffusion Models. In *Proceedings of ACM (GECCO-2026)*. ACM, New York, NY, USA, 7 pages. https://doi.org/10.475/123_4

1 Introduction

Estimation of Distribution Algorithms (EDAs) [1–3] are model-based evolutionary algorithms that replace traditional genetic operators with explicit probabilistic modeling. They learn a distribution from selected high-fitness solutions and sample new candidates from this model, thereby guiding the search toward promising regions of the solution space. A central challenge in EDA research is choosing a modeling approach that balances three criteria: expressiveness to capture variable dependencies, efficiency in learning from small datasets, and affordable sampling cost.

For discrete optimization, EDAs have employed models ranging from simple univariate distributions [4, 5] to complex probabilistic graphical models like Bayesian and Markov networks [6–8]. Recently, neural network-based generative models such as GANs [9]

and VAEs have been explored within EDAs [10–12], offering high expressiveness but often at a high computational cost.

Diffusion models (DMs) [13, 14] have emerged as state-of-the-art generative models, capable of modeling complex distributions through a gradual denoising process. Their success in continuous domains raises a compelling question: can they be effectively adapted to model search distributions in discrete combinatorial optimization? Applying DMs within EDAs presents unique challenges, including discrete data representation, computational efficiency per generation, and the need to bias sampling toward high-fitness regions.

Beyond the field of EAs, it is a relevant question in ML to investigate the limits of applicability of DMs, beyond the class of distributions where they are traditionally applied. For example, to investigate whether, in the context of search distributions, is possible to find a path from random distributions, or other initial distributions, to the target distribution of high quality solutions which are modeled in EDAs. Also, which information could this path linking these distributions provide about the characteristics of the optimization problem and the solutions search space. We argue that it is not only the potentially more accurate results provided by DMs what could be relevant for EDAs, but the understanding of the process of transforming the distributions itself since transitions to distributions that increase the probability of the optimal solutions has been an original goal in EDA research [15].

This paper makes several key contributions:

- We introduce two novel EDAs based on diffusion models: Diff-EDA (denoising-based) and DbD-EDA (deblending-based), both adapted for discrete optimization via Gumbel-Softmax and fitness guidance.
- We provide a comprehensive experimental evaluation on additive benchmarks and combinatorial problem instances (SAT, Ising, UBQP), comparing against classical EDAs (UMDA, Tree-EDA, EBNA, MN-FDAG) and a neural EDA baseline (C-VAE).
- We analyze the impact of algorithmic choices-loss functions, activation functions, and fitness guidance-on performance, offering insights into effective configurations for diffusion-based EDAs.
- We demonstrate that diffusion-based EDAs can achieve competitive or superior results compared to existing methods, particularly on problems with intricate dependencies, establishing a promising new research direction at the intersection of generative machine learning and evolutionary optimization.

The paper is organized as follows: Section 2 provides background on EDAs. Section 3 introduces diffusion models and the α -deblending variant. Sections 4 and 5 describe the proposed Diff-EDA and DbD-EDA algorithms. Section 6 reviews related work. Section 7 presents experimental results. Section 8 concludes and outlines future work.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

GECCO-2026,

© 2026 Copyright held by the owner/author(s).

ACM ISBN 123-4567-24-567/08/06

https://doi.org/10.475/123_4

2 Estimation of distribution algorithms

EDAs are a class of evolutionary algorithms that replace crossover and mutation with explicit probabilistic modeling. An EDA iterates through the following steps:

- (1) Initialization: Generate an initial population (often randomly).
- (2) Selection: Evaluate solutions and select a subset of high-fitness individuals.
- (3) Model Building: Learn a probabilistic model from the selected solutions.
- (4) Sampling: Generate new candidate solutions by sampling from the learned model.
- (5) Replacement: Form the next population by combining new samples with elite solutions.

This process repeats until a termination criterion is met. The core innovation lies in the model-building step, which captures variable interactions and guides exploration.

Explicitly modeling these relationships is particularly advantageous for problems with strong dependencies. This represents a fundamental difference from GAs, which implicitly exploit these relationships through the selection and recombination of building blocks. The rationale is that the most salient features of the best solutions, as captured by the probabilistic model, will be preserved in the newly generated solutions. Probabilistic model are also expected to capture characteristic patterns of the promising solutions by encoding relevant statistics about problem variables and their dependencies.

2.1 EDAs based on neural networks

When neural networks are used as models in EDAs [9–11], a variety of samplings methods can be applied. For typical NN generators such as generative adversarial nets (GANs) [16] and variational autoencoders (VAEs) [17], the corresponding methods for these models are used. GANs sample from a generator that has been trained to produce solutions that are indistinguishable from the training set (high quality solutions in this case), VAEs sample from a latent space, usually a multivariate Gaussian distribution, and transform these samples to the original problem representation (solution representation for EDAs). Some EDAs apply the conditional VAE (C-VAE) that allows to use information about the fitness function to condition the generation of new solutions [18].

There are also sampling methods used to generate solution from neural networks that are not generators. Among them is *Artificial Neural Network (ANN) Inversion* (Back-drive) [19]. Given a desired output value of a neural network, Back-drive determines the input values that would produce that output. This is applied in EDAs to sample from a multilayer perceptron (MLP) by setting the desired output (high fitness value) and inferring a genotype whose evaluation is likely to match the entered fitness.

In general, NN-based EDAs offer high model capacity but often require significant computational resources, which can be prohibitive in an iterative EDA framework.

3 Diffusion models

We briefly describe two types of diffusion models relevant to our work: denoising diffusion and diffusion-by-deblending.

3.1 Denoising Diffusion Models

Denoising diffusion models [13, 14, 20] consist of a forward process that gradually adds noise to data and a reverse process that learns to denoise, effectively generating samples from the data distribution.

3.1.1 Forward Process. Given a data sample $\mathbf{x}_0 \sim q(\mathbf{x}_0)$, Gaussian noise is added over T steps:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}), \quad (1)$$

where $\beta_t \in (0, 1)$ is a noise schedule. This allows direct sampling of $\mathbf{x}_t$ from $\mathbf{x}_0$:

$$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \quad (2)$$

with $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.

3.1.2 Reverse Process. A neural network $\epsilon_\theta(\mathbf{x}_t, t)$ is trained to predict the added noise by minimizing:

$$\mathbb{E}_{\mathbf{x}_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2]. \quad (3)$$

Sampling proceeds iteratively from $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$ using:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sqrt{\tilde{\beta}_t} \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}), \quad (4)$$

where $\tilde{\beta}_t = \frac{(1 - \bar{\alpha}_{t-1})\beta_t}{1 - \bar{\alpha}_t}$.

3.2 Diffusion-by-Deblending

The α -deblending method [21] learns a transformation between a source distribution p_0 and a target distribution p_1 . Given samples $\mathbf{x}_0 \sim p_0$ and $\mathbf{x}_1 \sim p_1$, a blended sample is constructed as $\mathbf{x} = (1 - \alpha)\mathbf{x}_0 + \alpha\mathbf{x}_1$ for $\alpha \in [0, 1]$. A network $D_\Theta(\mathbf{x}, \alpha)$ is trained to predict the difference $\mathbf{x}_1 - \mathbf{x}_0$ by minimizing:

$$\min_{\Theta} \mathbb{E}_{\alpha, \mathbf{x}_0, \mathbf{x}_1} [\|D_\Theta((1 - \alpha)\mathbf{x}_0 + \alpha\mathbf{x}_1, \alpha) - (\mathbf{x}_1 - \mathbf{x}_0)\|^2]. \quad (5)$$

Sampling starts from $\mathbf{x}_0 \sim p_0$ and iteratively applies the learned direction field:

$$\mathbf{x}_{\alpha_{t+1}} = \mathbf{x}_{\alpha_t} + (\alpha_{t+1} - \alpha_t) D_\Theta(\mathbf{x}_{\alpha_t}, \alpha_t). \quad (6)$$

3.3 Adaptations to discrete problems and neural network architecture considerations

3.3.1 Diffusion for binary data. The diffusion model algorithms introduced in the previous sections are originally defined for continuous variables. We are interested in the solution of discrete (binary) combinatorial problems. Therefore, the methods have to be adapted for discrete distribution. We use the Gumbel-Softmax distribution to implement learning in neural networks involving discrete variables. It was introduced by Jang et al. [22] to train VAEs and stochastic binary networks with discrete latent variables, allowing gradients to be computed via the reparameterization trick.

In our implementation, we follow a similar approach as Freivalds et al [23]. To make the sampling process differentiable with respect to the distribution parameters, the Gumbel-Softmax replaces the *argmax* by the *softmax* function with a temperature parameter τ . In practice, τ is often annealed from a high value to a low value during training to transition from smooth gradients to discrete-like samples.

3.3.2 Loss functions. The default loss function treats the problem as classification:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{NB} \sum_{j=1}^B \sum_{i=1}^n \left[x_{0,i}^{(j)} \log \hat{p}_{i,1}^{(j)} + (1 - x_{0,i}^{(j)}) \log \hat{p}_{i,0}^{(j)} \right] \quad (7)$$

where $\hat{p}_{i,c}^{(j)} = \text{softmax}(\mathbf{z}^{(j)})_{i,c}$ is the predicted probability.

Higher fitness samples receive higher weight in the loss:

$$w^{(j)} = \frac{f^{(j)} - f_{\min}}{f_{\max} - f_{\min} + \epsilon} \quad (8)$$

$$\mathcal{L}_{\text{weighted}} = \frac{1}{\sum_j w^{(j)}} \sum_{j=1}^B w^{(j)} \cdot \mathcal{L}_{\text{CE}}^{(j)} \quad (9)$$

The ranking loss prioritizes learning the relative ordering of solutions:

$$\mathcal{L}_{\text{rank}} = (1 - \lambda) \cdot \mathcal{L}_{\text{CE}} + \lambda \cdot \mathcal{L}_{\text{pairwise}} \quad (10)$$

where the pairwise ranking component is:

$$\mathcal{L}_{\text{pairwise}} = \frac{1}{|\mathcal{S}|} \sum_{(j,k) \in \mathcal{S}} \max(0, \ell^{(j)} - \ell^{(k)} + m) \quad (11)$$

where $\mathcal{S} = \{(j, k) : f^{(j)} > f^{(k)}\}$ is the set of valid pairs, $\ell^{(j)}$ is the per-sample loss for sample j , and m is the margin parameter.

The Huber loss provides robustness to outliers:

$$L_{\delta}(a) = \begin{cases} \frac{1}{2}a^2 & \text{if } |a| \leq \delta \\ \delta(|a| - \frac{1}{2}\delta) & \text{otherwise} \end{cases} \quad (12)$$

Applied to negative log-likelihood:

$$\mathcal{L}_{\text{Huber}} = \frac{1}{Bn} \sum_{j=1}^B \sum_{i=1}^n L_{\delta}(-\log \hat{p}_{i,x_{0,i}}^{(j)}) \quad (13)$$

The weighted MSE function assigns higher gradients to solutions with better fitness scores, forcing the model to prioritize reconstructing the best individuals in the population. The Ranking loss uses a margin-based loss (pairwise comparison) to ensure the model ranks variable probabilities correctly according to the fitness hierarchy of the training data. Finally, the Huber loss provides robustness against outliers in the training data (e.g., solutions that are selected but have significantly different configurations).

3.3.3 Sampling and fitness guidance. To bias the generation toward regions of higher fitness, we can incorporate fitness information into the reverse process. This is achieved by modeling the distribution of high-fitness individuals:

$$p_{\theta}(\mathbf{x}_0 | \mathbf{x}_t, t, f) \quad (14)$$

During sampling, the algorithm generates new candidates by starting from pure noise and iteratively denoising the solution:

$$\mathbf{x}_0 \sim p_{\theta}(\cdot | \mathbf{x}_T, T \rightarrow 0, \tilde{f}_{\text{target}}) \quad (15)$$

Fitness Guidance is similar to the use of fitness in C-VAE, it injects the fitness value f of solution $\mathbf{x}_0$ into the denoising network. This teaches the model the relationship between specific binary patterns and their objective function values.

4 Diffusion-by-denoising (Diff) EDA

Diff-EDA adapts discrete denoising diffusion for use within an EDA. Algorithm 1 outlines the training procedure. The forward process corrupts binary data using a categorical transition. The reverse process employs a neural network f_{θ} to predict the original clean data $\mathbf{x}_0$ from its noisy version $\mathbf{x}_t$. Gumbel-Softmax ensures differentiability. The model can be conditioned on fitness (fitness-guided variant) to steer sampling.

Algorithm 1 Diff-EDA Training

- 1: **Input:** Selected population $X \in \{0, 1\}^{N \times D}$, fitness values $F \in \mathbb{R}^N$
 - 2: **Hyperparameters:** Timesteps T , learning rate η , temperature τ , decay γ , loss type $\mathcal{L}_{\text{type}}$
 - 3: Initialize neural network f_{θ} , noise schedule $\beta_{1:T}$, compute $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$
 - 4: Normalize fitness: $\tilde{F} = \text{Normalize}(F)$
 - 5: **for** epoch = 1 to E **do**
 - 6: **for** each batch $(\mathbf{x}_0, f)$ in $(X, \tilde{F})$ **do**
 - 7: Sample $t \sim \text{Uniform}(\{1, \dots, T\})$
 - 8: **Forward:** Sample $\mathbf{x}_t \sim \text{Cat}(\mathbf{x}_t; \bar{\alpha}_t \mathbf{x}_0 + (1 - \bar{\alpha}_t)/K)$
 - 9: **Reverse:** Compute logits = $f_{\theta}(\mathbf{x}_t, t, f)$ (if fitness-guided) or $f_{\theta}(\mathbf{x}_t, t)$
 - 10: Compute loss $J(\theta)$ according to $\mathcal{L}_{\text{type}}$
 - 11: Update $\theta \leftarrow \theta - \eta \nabla_{\theta} J(\theta)$
 - 12: **end for**
 - 13: Anneal temperature: $\tau \leftarrow \tau \cdot \gamma$
 - 14: **end for**
 - 15: **Return:** Trained parameters θ
-

5 Diffusion-by-Deblending (DbD) EDA

DbD-EDA uses α -deblending to learn a transformation from a source distribution p_0 to a target distribution p_1 . Within the EDA, we set p_0 as the current population and p_1 as the selected population. For binary variables, blending is performed probabilistically: for each variable i ,

$$p(x_{\alpha,i} = 1 | \mathbf{x}_0, \mathbf{x}_1, \alpha) = (1 - \alpha)x_{0,i} + \alpha x_{1,i}. \quad (16)$$

The network D_{θ} learns to predict the difference $\mathbf{x}_1 - \mathbf{x}_0$. Algorithm 2 outlines the overall EDA loop incorporating DbD.

5.1 Adaptations to the algorithm

The alpha-deblending method, and in general, DMs require an original and target distributions in order to learn the forward and reverse diffusion paths. In most of applications, the target distribution is clearly defined. When applied as part of EDAs, both the target and source distributions can be selected in order to prioritize some aspect of the search. In DbD-EDA, p_0 is formed by solutions sampled from the {C}urrent population and p_1 comprises solutions sampled from the {S}elected population. The rationale behind this decision is the assumption that starting from the distribution of solutions in the current population is a better start than starting from a uniform distribution. The method can be directly related

Algorithm 2 DbD-EDA

Require: Fitness function f , population size N , generations G , selection ratio ρ

Require: DbD parameters: hidden_dims, epochs, batch_size, K , S

```

1: Initialize  $\mathcal{P}_0$  randomly, evaluate, store best  $\mathbf{x}^*$ 
2:  $P_{\text{prev}} \leftarrow \text{None}$ 
3: for generation  $g = 1$  to  $G$  do
4:   Select  $S_g$  (top  $\rho N$  from  $\mathcal{P}_{g-1}$ )
5:   Set  $P_0 \leftarrow \mathcal{P}_g$ ,  $P_1 \leftarrow S_g$ 
6:   Learn  $D_\theta \leftarrow \text{learn\_binary\_dbd}(P_0, P_1, \text{params})$ 
7:   Sample  $\mathcal{P}_g \leftarrow \text{sample\_dbd}(D_\theta, N, S_g, \text{samp\_params})$ 
8:   Evaluate  $\mathcal{P}_g$ , update  $\mathbf{x}^*$  if improved
9:    $P_{\text{prev}} \leftarrow \mathcal{P}_{g-1}$ 
10: end for
11: return  $\mathbf{x}^*$ ,  $f(\mathbf{x}^*)$ 

```

to truncated DMs [24]. At the time of sampling, starting from selected solutions intends to improve the fitness of these solutions, going beyond the quality of the solutions already explored.

We explore similar enhancements as in Diff-EDA: alternative loss functions (weighted-MSE, ranking, Huber) and fitness guidance by conditioning the network on fitness values. The idea of evaluating different loss function is based on the assumption that they can exhibit different properties for guiding the learning process. In DbD-EDA, the default loss function measures the squared error between predicted and actual differences. Therefore, the application of alternative loss functions is more experimental DbD-EDA.

6 Related Work

One promising domain for the application of diffusion models is optimization, particularly black-box (BB) optimization. Additionally, there is growing interest in using diffusion models to find optimal policies in reinforcement learning [25].

A limited number of recent works have proposed the use of diffusion models for black-box (BB) optimization, where the objective function can be computationally expensive. In this context, the goal of the diffusion model is to generate increasingly better solutions with improved objective values until the optimal solution is found. One approach to BB optimization using diffusion models is the Denoising Diffusion Optimization Models (DDOM) [26], which learns a conditional generative model over the domain of the BB function, conditioned on the function values, given an offline dataset. This approach aligns with previous applications of neural networks as surrogate functions [25].

In [27], the authors propose integrating diffusion models as a component of an expensive multi-objective Bayesian optimization algorithm (MOBO). Bayesian optimization is well-suited for this scenario, and various variants have been proposed for multi-objective optimization problems. The authors of [27] use the diffusion model to learn the distribution of the Pareto set. The dimensionality of the problems considered was relatively low (up to 10 decision variables), and the authors acknowledge that the relatively high computational cost of the algorithm makes it challenging to apply to very high-dimensional problems.

Yan and Jin [28] introduce *EmoDM*, an evolutionary multi-objective algorithm that incorporates a diffusion process component. The algorithm is closer to a transfer learning approach than to a typical evolutionary optimizer. The diffusion model is trained on a set of source multi-objective problems and is subsequently applied to solve a target problem. A distinctive feature of this approach is that the noise generated in the forward pass is derived from the evolution of an EA for the source instances. This noise corresponds to the differences between solutions in two consecutive populations, with the diffusion process proceeding in reverse order relative to the evolution. Specifically, the diffusion process starts from the last (converged) population and adds noise until it reaches the initial random population. The approach introduced in [28] also conditions the decision variables on the objective values. While this approach demonstrates the benefits of diffusion models within an evolutionary framework, it relies on the transfer of information among problem instances and is therefore fundamentally different from the EDA scenarios considered in this paper.

In [29], the authors propose integrating diffusion models as an offspring-generation process in EAs. Similar to an EDA, the idea is to sample solutions from high-fitness regions in the parameter space. As in [28], both decision variables and objectives are used to learn the model. A distinctive feature of this approach is the introduction of a weighting function in the loss function to bias the model toward high-fitness samples. The proposed algorithm is tested on two toy problems (double peaks and Rastrigin) and the cart-pole reinforcement learning problem. The authors conclude that the introduced method produces significant improvements in adaptability to changing environments while maintaining reliable convergence to target solutions.

7 Experiments

The goal of our experiments is to evaluate the accuracy and efficiency of the EDAs based on diffusion models to solve different types of combinatorial problems. In addition, we evaluate whether different enhancements introduced to the learning and sampling procedures of the diffusion models has a positive impact in the optimization process. In this section, we first introduce the experimental framework including the function benchmark, EDAs used in the comparison and the criteria used to evaluate the algorithms. Then we present the numerical results and the discussion.

7.1 Experimental framework

We evaluate the accuracy and efficiency of Diff-EDA and DbD-EDA on additive functions (OneMax, Deceptive3, kDeceptive3, kDeceptive5, HIFF) and combinatorial problems (5 SAT instances, 4 Ising instances, 1 UBQP instance).

Since our main focus is to determine whether modeling the distributions by DMs can provide an advantage over other probabilistic modeling techniques used in EDA, we compare Diff-EDA and DbD-EDA with EDAs based on probabilistic graphical models. While there exist more sophisticated EDAs (e.g., Hierarchical Bayesian Optimization Algorithm (hBOA) [30]) and alternative methods that efficiently exploit the structure of interactions among the variables (e.g., Gene-pool Optimal Mixing Evolutionary Algorithm (GOMEA) [31]), we select EDAs that learn a single probabilistic

graphical model from which a factorization of the distribution can be easily derived. For the same reason, we do not use local optimization methods that have been proved to improve the results of the search when combined with of EDAs.

- Univariate Marginal Distribution Algorithm (UMDA) [1]: It is the simplest EDA that models the distribution as a product of univariate marginal distributions.
- Tree-EDA [32, 33]: Learns a tree-structure of dependencies where each variable can depend at most on another variable. Mutual information is used to learn the dependencies.
- Estimation of Bayesian Network Algorithm (EBNA) [6]: A Bayesian network is used to represent the distribution of the selected solutions.
- Markov Network FDA (MN-FDAG) [34]: Learns an approximate junction tree describing the most important dependencies as captured by the mutual information.

In addition, as part of the comparison we evaluate the performance of VAE-EDA since this is perhaps the most advanced EDA based on generative models.

For all EDAs we use a population size $N = 100 \times n$ with a 250 generations. We used truncation selection with truncation parameter $\rho = 0.5$ for additive functions and $\rho = 0.1$ for the combinatorial problems. Elitism was implemented and best solution in each generation is passed to the next. For all methods, we evaluated in preliminary experiments that a mutation step was useful to avoid early convergence and added this step. The mutation is applied when for any of the variables in the newly generated population, the frequency of one of the values goes beyond 95%. In that case, for all variables for which the frequency reach the threshold, a number of solutions with the majority value are flipped to guarantee a minimum 5% of the minority value for each variable.

After preliminary tuning, we set default configurations:

- Diff-EDA: Ranking loss, ReLU activation, no fitness guidance.
- DbD-EDA: Huber loss, ReLU activation, fitness guidance enabled.
- C-VAE: ReLU for encoder/decoder.

These configurations are not necessarily the optimal for all problems, and we investigate this question in section ???. We compare the algorithms in terms of the number of times the best solution has been found, the average fitness of best solutions, and the number of generations until best solution is found.

We measure performance by best fitness found, success rate, and generations to convergence.

7.2 Comparisson to other EDAs

7.2.1 Results for additive functions.

7.2.2 Results for combinatorial problems. Table 3 summarizes the mean best fitness on combinatorial instances. Diff-EDA achieves the best results on most SAT and UBQP instances, while traditional EDAs perform well on Ising problems. DbD-EDA is competitive, often close to Diff-EDA. C-VAE lags behind the diffusion-based approaches.

7.3 Impact of configurations in performance

One of the results of our analysis is that the choice of the loss function and the activation function, together with the application of fitness guidance, have an impact on the search and can significantly improve the results of EDAs that use DMs. Table ?? shows the best configurations for EDAs based on NNs and the mean fitness. For Diff-EDA, the ranking loss consistently performs well. For DbD-EDA, Huber and MSE losses are effective. Fitness guidance is beneficial for DbD-EDA but less critical for Diff-EDA in these experiments.

We further compare the impact of the configurations in terms of the best fitness for the two algorithms. Figure ?? shows two heatmap figures illustrating the average fitness computed from all SAT instances taken together for each pair (loss, activation) of Diff-EDA (with not fitness guidance) and DbD-EDA (using final guidance). Similar information is presented in the supplementary information for the other two combinatorial problems.

7.4 Average number of generations and computational time

8 Conclusions

We have introduced Diff-EDA and DbD-EDA as algorithms that follow a different approach to the traditional EDAs to learn the distributions. We have shown that the algorithms can produce competitive results for combinatorial problems and presented for the first time an analysis of the impact of loss and activation functions in the performance of EDAs based on DMs. A limitation of EDAs based on DMs is the computational time needed for learning the models. We envision the application of these algorithms for computationally costly fitness functions or problems with intricate dependencies. In addition, active research is being conducted to reduce the number of steps required for generating samples from diffusion models [14, 35, 36].

Our analysis highlights the importance of algorithmic choices such as loss functions and activation functions, with ranking and Huber losses showing strong performance. While computational cost remains a challenge, the expressiveness and flexibility of diffusion models offer a promising new direction for EDAs.

Future work includes:

- Investigating the relationship between problem structure and optimal diffusion model configurations.
- Analyzing the quality of intermediate solutions along the diffusion path to better understand the search dynamics.
- Incorporating faster sampling techniques to improve scalability.
- Extending the approach to higher-dimensional and mixed-integer problems.

By successfully integrating diffusion models into the EDA framework, this work opens a new avenue for leveraging advanced generative models in evolutionary optimization.

Functio	Diff-EDA			DbD-CS			UMDA			TreeEDA			EBNA			MN-FDAG			C-VAE		
	Succ.	Fitness	Gen.	Succ.	Fitness	Gen.	Succ.	Fitness	Gen.	Succ.	Fitness	Gen.	Succ.	Fitness	Gen.	Succ.	Fitness	Gen.	Succ.	Fitness	Gen.
OneMax	1.00	30.00	5.35	1.00	30.00	7.95	1.00	30.00	4.45	1.00	30.00	5.35	1.00	30.00	5.10	1.00	30.00	4.80	1.00	30.00	3.95
KDeceptive3	0.00	27.65	132.20	0.50	29.45	132.85	0.50	29.35	14.85	1.00	30.00	6.25	1.00	30.00	5.75	1.00	30.00	6.65	0.95	29.95	11.25
Deceptive3	0.00	9.34	103.90	0.15	9.84	140.15	0.00	9.14	4.65	0.80	9.96	15.25	0.55	9.95	7.95	0.00	9.74	13.35	0.20	9.86	12.85
HIFF	0.00	188.40	142.05	0.00	313.40	137.35	0.00	285.20	26.30	1.00	448.00	12.00	1.00	448.00	11.15	0.00	331.30	23.30	0.20	340.80	36.15
KDeceptive5	0.00	24.30	21.90	0.05	25.15	46.30	0.00	24.00	5.60	0.05	25.65	8.95	0.00	26.30	7.55	0.00	24.90	7.65	0.00	27.45	10.30

Table 1: Comparison of Algorithm Performance (Success Rate, Best Fitness, and Generation)

Function	Conf.	Diff-EDA			Conf.	DbD-CS			Conf.	C-VAE		
		Succ.	Fitness	Gen.		Succ.	Fitness	Gen.		Succ.	Fitness	Gen.
KDeceptive3	[E, Ra, Y]	1.00	30.00	11.70	[T, WM, Y]	1.00	30.00	60.75	[R/R]	0.95	29.95	11.25
Deceptive3	[E, WM, N]	0.05	9.87	122.80	[T, WM, Y]	0.65	9.94	145.15	[R/R]	0.20	9.86	12.85
HIFF	[T, Ra, N]	1.00	448.00	50.15	[R, Ra, N]	0.70	414.90	106.60	[R/R]	0.20	340.80	36.15
KDeceptive5	[E, WM, N]	0.35	28.45	27.55	[T, H, Y]	0.25	28.05	130.40	[T/E]	0.10	27.85	138.75

Table 2: Absolute best performance and associated configurations. Act: R=Relu, E=Elu, T=Tanh; Loss: H=Huber, Ra=Rank, M=MSE, WM=Weighted MSE; FG (Fitness Guided): Y=Yes, N=No. C-VAE config shown as [Enc/Dec].

instance	Diff-EDA	DbD-CS	UMDA	TreeEDA	EBNA	MN-FDAG	C-VAE
uf100-01	427.65	427.45	426.00	416.90	426.55	426.40	426.20
uf100-02	428.40	428.20	427.80	416.30	427.60	428.20	428.30
uf100-03	427.80	427.95	427.25	416.85	427.20	427.20	427.50
uf100-04	428.60	428.45	427.75	415.50	427.75	428.00	428.15
uf100-05	428.80	427.95	427.75	416.35	427.75	427.95	428.25
SG_100_1	128.80	126.00	123.20	113.40	125.20	124.40	125.40
SG_100_2	135.20	135.20	128.20	119.60	131.40	131.60	131.20
SG_100_3	136.20	136.00	129.80	117.00	132.40	128.80	131.40
SG_100_4	130.60	128.80	121.80	115.40	127.20	125.80	125.40
bqp100	3894.30	3885.90	3858.65	2425.00	3882.25	3853.20	3877.55

Table 3: Comparison of Best Fitness across SAT, Ising, and UBQP ($\alpha = 0.95$)

Acknowledgments

We have used Claude and Copilot AI programming assistants as a support for code design and implementation for the Experimental section. Gemini and Deepseek AI assistants have been used for grammar and style correction of the paper.

References

- [1] H. Mühlenbein and G. Paaß, "From recombination of genes to the estimation of distributions I. Binary parameters," in *Parallel Problem Solving from Nature - PPSN IV*, ser. Lectures Notes in Computer Science, H.-M. Voigt, W. Ebeling, I. Rechenberg, and H.-P. Schwefel, Eds., vol. 1141. Berlin: Springer, 1996, pp. 178–187.
- [2] S. Baluja and S. Davies, "Using optimal dependency-trees for combinatorial optimization: Learning the structure of the search space," in *Proceedings of the 14th International Conference on Machine Learning*, D. H. Fisher, Ed. San Francisco, CA.: Morgan Kaufmann, 1997, pp. 30–38.
- [3] P. Larrañaga, R. Etxeberria, J. A. Lozano, and J. M. Peña, "Optimization by learning and simulation of Bayesian and Gaussian networks," Department of Computer Science and Artificial Intelligence, University of the Basque Country, Technical Report EHU-KZAA-IK-4/99, 1999.
- [4] C. González, J. A. Lozano, and P. Larrañaga, "Mathematical modeling of UMDAc algorithm with tournament selection. Behaviour on linear and quadratic functions," *International Journal of Approximate Reasoning*, vol. 31, no. 4, pp. 313–340, 2002.
- [5] Z. G. Arenas, J. C. Jimenez, L.-V. Lozada-Chang, and R. Santana, "Estimation of distribution algorithms for the computation of innovation estimators of diffusion processes," *Mathematics and Computers in Simulation*, vol. 187, pp. 449–467, 2021. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0378475421000926>
- [6] R. Etxeberria and P. Larrañaga, "Global optimization using Bayesian networks," in *Proceedings of the Second Symposium on Artificial Intelligence (CIMA-99)*, A. Ochoa, M. R. Soto, and R. Santana, Eds. Editorial Academia. Havana, Cuba, 1999, pp. 332–339.
- [7] M. Pelikan, D. E. Goldberg, and E. Cantú-Paz, "BOA: The Bayesian optimization algorithm," in *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-1999*, W. Banzhaf, J. Daida, A. E. Eiben, M. H. Garzon, V. Honavar, M. Jakiela, and R. E. Smith, Eds., vol. I. Orlando, FL: Morgan Kaufmann Publishers, San Francisco, CA, 1999, pp. 525–532.
- [8] S. Shakya and J. McCall, "Optimization by estimation of distribution with DEUM framework based on Markov random fields," *International Journal of Automation and Computing*, vol. 4, no. 3, pp. 262–272, 2007.
- [9] M. Probst, "Generative adversarial networks in estimation of distribution algorithms for combinatorial optimization," *CoRR*, vol. abs/1509.09235, 2015. [Online]. Available: <http://arxiv.org/abs/1509.09235>
- [10] S. Bhattacharjee and R. Gras, "Estimation of distribution using population queue based variational autoencoders," in *2019 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2019, pp. 1406–1414.
- [11] C. He, S. Huang, R. Cheng, K. C. Tan, and Y. Jin, "Evolutionary multiobjective optimization driven by generative adversarial networks (GANs)," *IEEE Transactions on Cybernetics*, vol. 51, no. 6, pp. 3129–3142, 2020.
- [12] B. Zhang, F. Sun, J. Tian, L. Wang, C. Zhang, S. Liu, and B. Yang, "OPT-VAE: A variational autoencoder-based estimation of distribution algorithm for continuous optimization," *IEEE Transactions on Evolutionary Computation*, 2026.
- [13] J. Ho, A. Jain, and P. Abbeel, "Denosing diffusion probabilistic models," *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851, 2020.
- [14] J. Song, C. Meng, and S. Ermon, "Denosing diffusion implicit models," *CoRR*, vol. abs/2010.02502, 2020. [Online]. Available: <http://arxiv.org/abs/2010.02502>
- [15] H. Mühlenbein and T. Mahnig, "Evolutionary algorithms and the Boltzmann distribution," in *Foundation of Genetic Algorithms 7*, K. A. DeJong, R. Poli, and J. Rowe, Eds. Morgan Kaufmann, 2002, pp. 133–150.

Instance	Type	DiffEDA	DbD-CS	C-VAE
uf100-01	Config	[R, RK, N]	[R, M, Y]	[R/R]
	Fitness	427.65	427.50	426.20
uf100-02	Config	[R, RK, Y]	[R, H, Y]	[R/R]
	Fitness	428.55	428.20	428.30
uf100-03	Config	[E, RK, N]	[R, H, Y]	[R/R]
	Fitness	427.95	427.95	427.50
uf100-04	Config	[E, RK, N]	[R, H, Y]	[E/R]
	Fitness	428.70	428.45	428.15
uf100-05	Config	[R, RK, N]	[R, RK, Y]	[R/R]
	Fitness	428.80	428.20	428.25
SG_100_1	Config	[R, WM, N]	[R, M, Y]	[R/R]
	Fitness	129.20	129.60	125.40
SG_100_2	Config	[R, RK, N]	[R, M, Y]	[T/R]
	Fitness	135.20	136.00	131.40
SG_100_3	Config	[R, RK, N]	[R, H, Y]	[T/R]
	Fitness	136.20	136.00	132.40
SG_100_4	Config	[R, RK, N]	[R, M, Y]	[T/R]
	Fitness	130.60	129.20	126.80
bqp100	Config	[E, RK, N]	[T, H, Y]	[T/R]
	Fitness	3903.20	3903.60	3890.65

Table 4: Best fitness and corresponding configurations [Activation, Loss, Fitness Guided]. Acronyms: R (ReLU), E (ELU), T (Tanh), M (MSE), WM (Weighted MSE), RK (Ranking), H (Huber), Y/N (Yes/No for Fitness Guided), VAE (Standard VAE Loss).

- [16] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *Advances in neural information processing systems*, 2014, pp. 2672–2680.
- [17] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," *CoRR*, vol. abs/1312.6114, 2013. [Online]. Available: <http://arxiv.org/abs/1312.6114>
- [18] U. Garcıarena, R. Santana, and A. Mendiburu, "Expanding variational autoencoders for learning and exploiting latent representations in search distributions," in *Proceedings of the 2018 on Genetic and Evolutionary Computation Conference*. ACM, 2018, pp. 849–856. [Online]. Available: <https://dl.acm.org/doi/10.1145/3205455.3205645>
- [19] S. Baluja, "Learning deep models of optimization landscapes," in *2017 IEEE Symposium Series on Computational Intelligence (SSCI)*. IEEE, 2017, pp. 1–10.
- [20] M. Chen, S. Mei, J. Fan, and M. Wang, "An overview of diffusion models: Applications, guided generation, statistical rates and optimization," *CoRR*, vol. abs/2404.07771, 2024. [Online]. Available: <http://arxiv.org/abs/2404.07771v1>
- [21] E. Heitz, L. Belcour, and T. Chambon, "Iterative α -(de) blending: A minimalist deterministic diffusion model," in *ACM SIGGRAPH 2023 Conference Proceedings*, 2023, pp. 1–8.
- [22] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-softmax," *CoRR*, vol. abs/1611.01144, 2016. [Online]. Available: <http://arxiv.org/abs/1611.01144>
- [23] K. Freivalds, E. Ozoliņš, and G. Bārzdīņš, "Discrete denoising diffusion approach to integer factorization," in *International Conference on Artificial Neural Networks*. Springer, 2023, pp. 123–134.
- [24] H. Zheng, P. He, W. Chen, and M. Zhou, "Truncated diffusion probabilistic models," *CoRR*, vol. abs/2202.09671, 2022. [Online]. Available: <http://arxiv.org/abs/2202.09671>
- [25] Z. Zhu, H. Zhao, H. He, Y. Zhong, S. Zhang, H. Guo, T. Chen, and W. Zhang, "Diffusion models for reinforcement learning: A surveyuniversal adversarial audio perturbations," *CoRR*, vol. abs/2311.01223, 2024. [Online]. Available: <http://arxiv.org/abs/2311.01223v4>
- [26] S. Krishnamoorthy, S. Mashkaria, and A. Grover, "Diffusion models for black-box optimization," in *Proceedings of the 40th International Conference on Machine Learning*, Honolulu, Hawaii, 2023, p. 1–16.
- [27] B. Li, Z. Di, Y. Lu, H. Qian, F. Wang, P. Yang, K. Tang, and A. Zhou, "Expensive multi-objective Bayesian optimization based on diffusion models," *CoRR*, vol. abs/2405.08674, 2024. [Online]. Available: <http://arxiv.org/abs/2405.08674>
- [28] X. Yan and Y. Jin, "EmoDM: A diffusion model for evolutionary multi-objective optimization," *CoRR*, vol. abs/2401.15931, 2024. [Online]. Available: <http://arxiv.org/abs/2401.15931>
- [29] B. Hartl, Y. Zhang, H. Hazan, and M. Levin, "Heuristically adaptive diffusion-model evolutionary strategy," *CoRR*, vol. abs/2411.13420, 2024. [Online]. Available: <http://arxiv.org/abs/2411.13420>
- [30] M. Pelikan, D. E. Goldberg, and E. Cantú-Paz, "Linkage problem, distribution estimation, and Bayesian networks," *Evolutionary Computation*, vol. 8, no. 3, pp. 311–340, 2000.
- [31] D. Thierens and P. A. Bosman, "Optimal mixing evolutionary algorithms," in *Proceedings of the Genetic and Evolutionary Computation Conference GECCO-2011*, 2011, pp. 617–624.
- [32] J. S. De Bonet, C. L. Isbell, and P. Viola, "MIMIC: Finding optima by estimating probability densities," in *Advances in Neural Information Processing Systems*, M. C. Mozer, M. I. Jordan, and T. Petsche, Eds., vol. 9. The MIT Press, Cambridge, 1997, pp. 424–430.
- [33] M. Pelikan and H. Mühlenbein, "The bivariate marginal distribution algorithm," in *Advances in Soft Computing - Engineering Design and Manufacturing*, R. Roy, T. Furuhashi, and P. Chawdhry, Eds. London: Springer, 1999, pp. 521–535.
- [34] A. Ochoa, M. R. Soto, R. Santana, J. Madera, and N. Jorge, "The factorized distribution algorithm and the junction tree: A learning perspective," in *Proceedings of the Second Symposium on Artificial Intelligence (CIMA-99)*, A. Ochoa, M. R. Soto, and R. Santana, Eds. Havana, Cuba: Editorial Academia. Havana, Cuba, March 1999, pp. 368–377.
- [35] C. Lu, S. Li, and Z. Lu, "Building energy prediction using artificial neural networks: A literature survey," *Energy and Buildings*, vol. 262, p. 111718, 2022.
- [36] Y. Ren, H. Chen, Y. Zhu, W. Guo, Y. Chen, G. M. Rotskoff, M. Tao, and L. Ying, "Fast solvers for discrete diffusion models: Theory and applications of high-order algorithms," *CoRR*, vol. abs/2502.00234, 2025. [Online]. Available: <http://arxiv.org/abs/2502.00234>